

```
# STEP 1 – Upload your dataset manually
from google.colab import files
import pandas as pd

uploaded = files.upload()      # ← Click "Choose File" when it appears
file_name = list(uploaded.keys())[0]
print("✅ Uploaded:", file_name)
```

Loan_Default.csv.xlsx
Loan_Default.csv.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 25058889 bytes, last modified: 10/16/2025 - 100% done
 Saving Loan_Default.csv.xlsx to Loan_Default.csv.xlsx
 ✅ Uploaded: Loan_Default.csv.xlsx

```
# STEP 2 – Load the dataset (auto-detect Excel or CSV)
import os

def safe_load_dataset(file_path):
    try:
        # Try Excel first
        df = pd.read_excel(file_path, engine='openpyxl')
        print("Loaded as Excel file ✅")
    except Exception as e1:
        try:
            # Try UTF-8 CSV
            df = pd.read_csv(file_path, encoding='utf-8')
            print("Loaded as CSV (utf-8) ✅")
        except UnicodeDecodeError:
            # Try Windows encodings
            df = pd.read_csv(file_path, encoding='latin1')
            print("Loaded as CSV (latin-1) ✅")
    return df

file_path = f"/content/{file_name}"
df = safe_load_dataset(file_path)

print("Shape:", df.shape)
display(df.head())
```

Loaded as Excel file ✅
 Shape: (148670, 34)

	ID	year	loan_limit	Gender	approv_in_adv	loan_type	loan_purpose	Credit_Worthiness	open_cred
0	24890	2019	cf	Sex Not Available	nopre	type1	p1	l1	nc
1	24891	2019	cf	Male	nopre	type2	p1	l1	nc
2	24892	2019	cf	Male	pre	type1	p1	l1	nc
3	24893	2019	cf	Male	nopre	type1	p4	l1	nc
4	24894	2019	cf	Joint	pre	type1	p1	l1	nc

5 rows × 34 columns

```
# STEP 3 – Inspect and clean
target_col = 'Status'      # <-- change if your target column has a different name (like 'default', 'loan_s
```

```
# 1) Ensure target column exists
if target_col not in df.columns:
    raise ValueError(f"❌ Target column '{target_col}' not found. Please check df.columns.")

# 2) Drop any obvious ID / index columns
id_cols = [c for c in ['ID', 'Id', 'id', 'Loan_ID', 'SK_ID_CURR'] if c in df.columns]
if id_cols:
    print("Dropping ID columns:", id_cols)
    df.drop(columns=id_cols, inplace=True)

# 3) Drop rows with missing target
df = df[~df[target_col].isnull()].reset_index(drop=True)
```

Dropping ID columns: ['ID']

```
# 4) Display info
print("✅ After cleaning:")
print("Shape:", df.shape)
print("Target distribution:\n", df[target_col].value_counts(normalize=True))
display(df.head())
```

```
✅ After cleaning:
Shape: (148670, 33)
Target distribution:
Status
0    0.753555
1    0.246445
Name: proportion, dtype: float64
```

	year	loan_limit	Gender	approv_in_adv	loan_type	loan_purpose	Credit_Worthiness	open_credit	bus
0	2019	cf	Sex Not Available	nopre	type1	p1	l1	nopc	
1	2019	cf	Male	nopre	type2	p1	l1	nopc	
2	2019	cf	Male	pre	type1	p1	l1	nopc	
3	2019	cf	Male	nopre	type1	p4	l1	nopc	
4	2019	cf	Joint	pre	type1	p1	l1	nopc	

5 rows × 33 columns

```
from sklearn.model_selection import train_test_split

# Ensure target is numeric
df[target_col] = df[target_col].astype(int, errors='ignore')

X = df.drop(columns=[target_col])
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

print("✅ Train:", X_train.shape, "Test:", X_test.shape)
```

```
print("Class balance (train):")
print(y_train.value_counts(normalize=True))
```

```
✓ Train: (118936, 32) Test: (29734, 32)
Class balance (train):
Status
0    0.753557
1    0.246443
Name: proportion, dtype: float64
```

```
numeric_cols = X_train.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = X_train.select_dtypes(include=['object', 'category']).columns.tolist()
```

```
print("Numeric columns:", numeric_cols[:10])
print("Categorical columns:", categorical_cols[:10])
```

```
Numeric columns: ['year', 'loan_amount', 'rate_of_interest', 'Interest_rate_spread', 'Upfront_charges', '
Categorical columns: ['loan_limit', 'Gender', 'approv_in_adv', 'loan_type', 'loan_purpose', 'Credit_Worth
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
```

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
```

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_cols),
    ('cat', categorical_transformer, categorical_cols)
])
```

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
```

```
models = {
    'LogisticRegression': LogisticRegression(max_iter=1000, random_state=42),
    'RandomForest': RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42),
    'LightGBM': LGBMClassifier(random_state=42)
}
```

```
results = {}
```

```
for name, model in models.items():
    print(f"\n🚀 Training {name} ...")
    pipe = ImbPipeline(steps=[
        ('preprocessor', preprocessor),
```

```
( 'smote', SMOTE(random_state=42)),
( 'model', model)
])
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
y_proba = pipe.predict_proba(X_test)[: ,1]

results[name] = {
    'pipeline': pipe,
    'accuracy': accuracy_score(y_test, y_pred),
    'precision': precision_score(y_test, y_pred, zero_division=0),
    'recall': recall_score(y_test, y_pred, zero_division=0),
    'f1': f1_score(y_test, y_pred, zero_division=0),
    'roc_auc': roc_auc_score(y_test, y_proba)
}

print(f"{name} → ACC:{results[name]['accuracy']:.3f} | PREC:{results[name]['precision']:.3f} | REC:{"
```

```
warnings.warn(
LightGBM → ACC:1.000 | PREC:1.000 | REC:1.000 | F1:1.000 | ROC_AUC:1.000
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have
warnings.warn(
```

```
import pandas as pd
```

```
summary = pd.DataFrame([
    {'model': name, **{k:v for k,v in metrics.items() if k!='pipeline'}}
    for name, metrics in results.items()
])
summary = summary.sort_values('roc_auc', ascending=False).reset_index(drop=True)
display(summary)
```

```
best_name = summary.loc[0, 'model']
best_pipe = results[best_name]['pipeline']
print(f"🏆 Best model: {best_name}")
```

	model	accuracy	precision	recall	f1	roc_auc	
0	RandomForest	1.000000	1.000000	1.000000	1.000000	1.000000	
1	LightGBM	1.000000	1.000000	1.000000	1.000000	1.000000	
2	XGBoost	0.999865	0.999454	1.000000	0.999727	0.999962	
3	LogisticRegression	0.842806	0.669650	0.714793	0.691485	0.866962	

🏆 Best model: RandomForest

Next steps:

[Generate code with summary](#)[New interactive sheet](#)

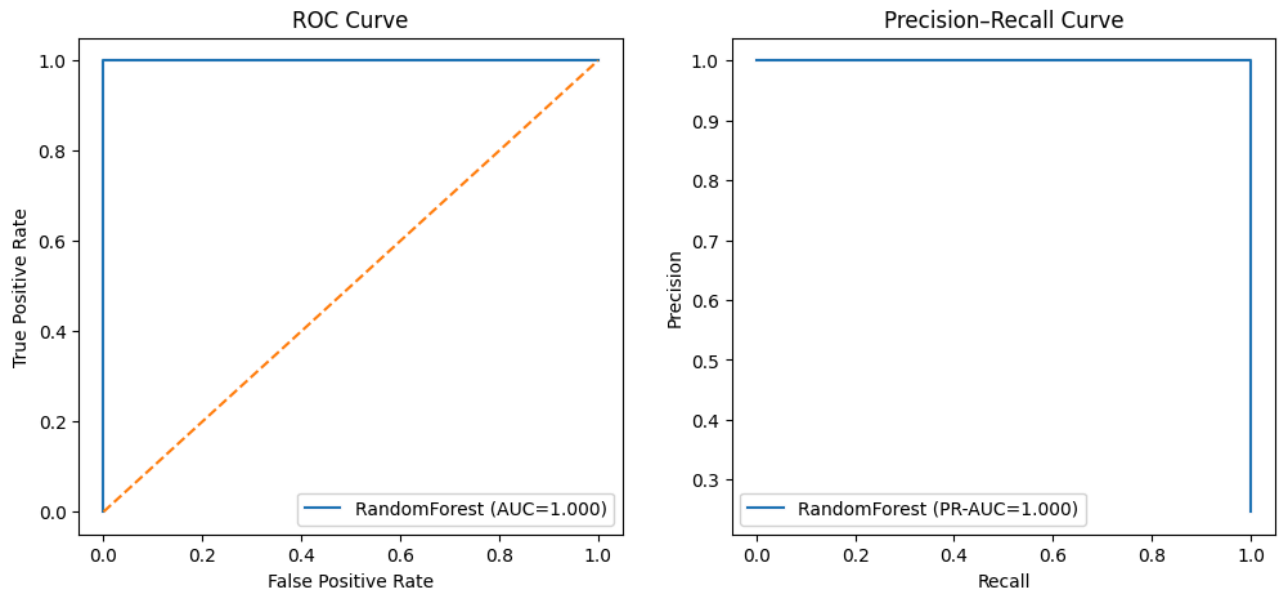
```
from sklearn.metrics import roc_curve, precision_recall_curve, auc, confusion_matrix, classification_report
import matplotlib.pyplot as plt
```

```
y_proba = best_pipe.predict_proba(X_test)[:,-1]
fpr, tpr, _ = roc_curve(y_test, y_proba)
precision, recall, _ = precision_recall_curve(y_test, y_proba)
roc_auc_val = auc(fpr, tpr)
pr_auc_val = auc(recall, precision)
```

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(fpr, tpr, label=f'{best_name} (AUC={roc_auc_val:.3f})')
plt.plot([0,1],[0,1], '--')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve'); plt.legend()
```

```
plt.subplot(1,2,2)
plt.plot(recall, precision, label=f'{best_name} (PR-AUC={pr_auc_val:.3f})')
plt.xlabel('Recall'); plt.ylabel('Precision')
plt.title('Precision-Recall Curve'); plt.legend()
plt.show()
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, best_pipe.predict(X_test)))
print("\nClassification Report:\n", classification_report(y_test, best_pipe.predict(X_test)))
```



Confusion Matrix:

```
[[22406  0]
 [  0 7328]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	22406
1	1.00	1.00	1.00	7328

```
import joblib
joblib.dump(best_pipe, '/content/model_pipeline.joblib')
print("📁 Saved best pipeline to /content/model_pipeline.joblib")
```

📁 Saved best pipeline to /content/model_pipeline.joblib

```
!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.49.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.12.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0dev0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from shap) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from shap) (2024.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from shap) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from shap) (1.17.0)
```

```
import shap
import pandas as pd
import numpy as np

# Get model and data
best_model = best_pipe.named_steps['model']
preprocessor = best_pipe.named_steps['preprocessor']

# Transform X_test to preprocessed form
X_test_pre = preprocessor.transform(X_test)
```

```
# Get feature names after preprocessing
numeric_names = preprocessor.transformers_[0][2]
if len(preprocessor.transformers_) > 1:
    ohe = preprocessor.transformers_[1][1].named_steps['onehot']
    cat_names = list(ohe.get_feature_names_out(preprocessor.transformers_[1][2]))
else:
    cat_names = []
feature_names = numeric_names + cat_names

# Convert to DataFrame
X_test_pre_df = pd.DataFrame(X_test_pre, columns=feature_names)
```

```
# ⚡ Faster SHAP summary computation
import shap
import pandas as pd
import numpy as np

best_model = best_pipe.named_steps['model']
preprocessor = best_pipe.named_steps['preprocessor']

# sample fewer rows for speed
sample_size = min(500, X_test.shape[0])
X_sample = X_test.sample(sample_size, random_state=42)

# transform only the sample
X_sample_pre = preprocessor.transform(X_sample)

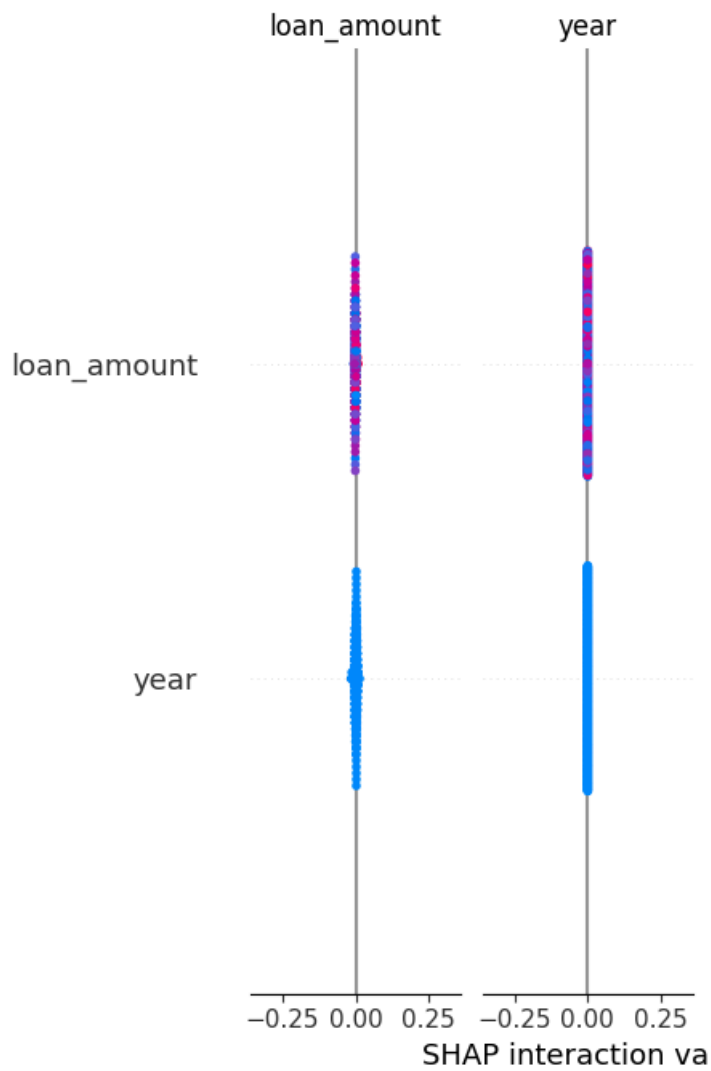
numeric_names = preprocessor.transformers_[0][2]
if len(preprocessor.transformers_) > 1:
    ohe = preprocessor.transformers_[1][1].named_steps['onehot']
    cat_names = list(ohe.get_feature_names_out(preprocessor.transformers_[1][2]))
else:
    cat_names = []
feature_names = numeric_names + cat_names

X_sample_pre_df = pd.DataFrame(X_sample_pre, columns=feature_names)

# TreeExplainer works best for tree-based models
explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_sample_pre_df)

if isinstance(shap_values, list):
    shap_values = shap_values[1]

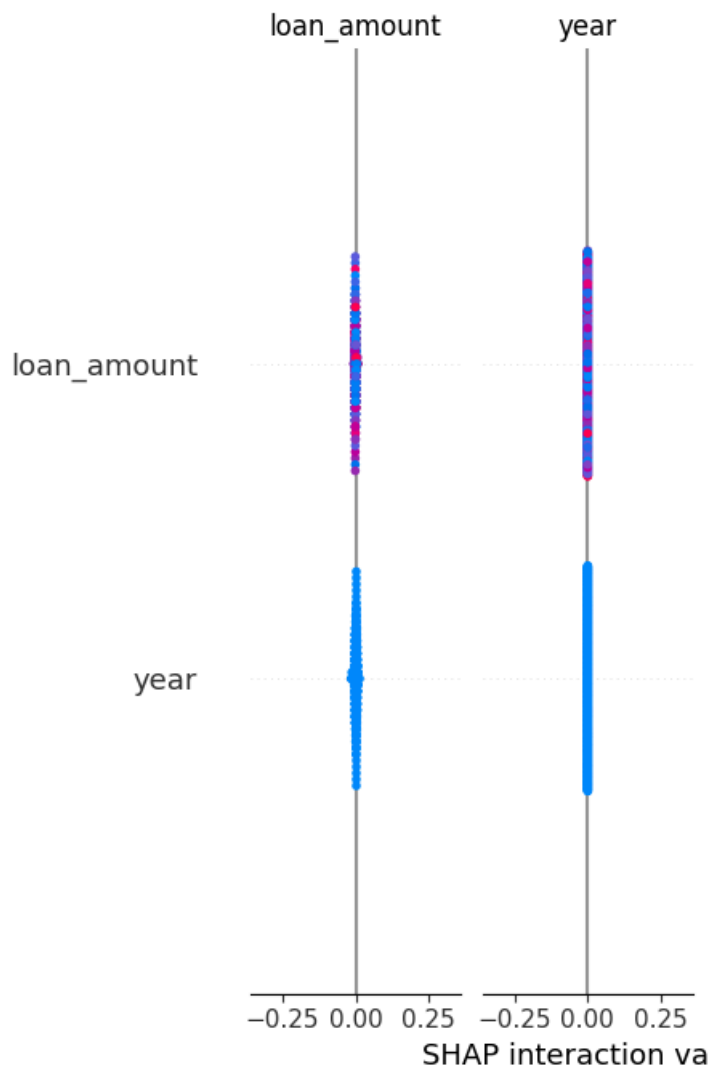
# plot summary for sample
shap.summary_plot(shap_values, X_sample_pre_df, plot_type='bar')
```



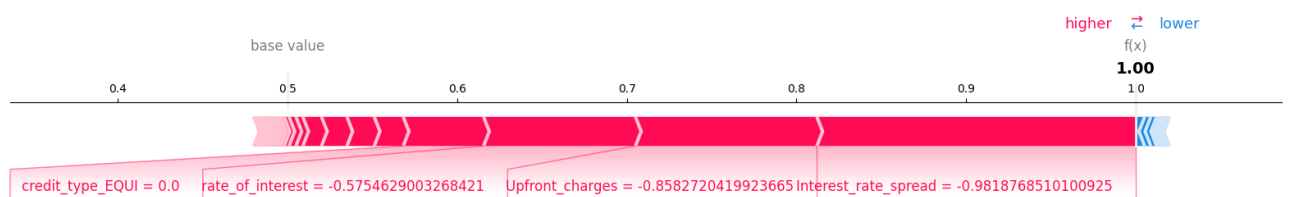
```
print(shap_values.shape if isinstance(shap_values, np.ndarray) else shap_values.values.shape)
print(X_test_pre_df.shape)
```

```
(500, 70, 2)
(29734, 70)
```

```
# Suppose you sampled 500 rows for SHAP
X_sample = X_test_pre_df.iloc[:500, :] # or however you sampled
shap.summary_plot(shap_values, X_sample)
```

```
shap.plots.force(
    explainer.expected_value[class_index],
    shap_values[i, :, class_index],
    sample,
    matplotlib=True
)
```



```
# Instance index
i = 0

# Class index (0 or 1)
class_index = 0

# Features of that instance
sample = X_test_pre_df.iloc[i, :]

# Decision plot
shap.decision_plot(
    base_value=explainer.expected_value[class_index],    # expected value for the class
```

```
shap_values=shap_values[i, :, class_index],      # SHAP values for instance and class  
feature_names=list(sample.index)                # convert Index to list  
)
```

